WORKSHOP 2569 · วันที่ 1 จาก 2

Docker สู่ CI/CD

Docker Fundamentals — จาก “รันได้บนเครื่องผม”
สู่ image ที่พร้อมขึ้น pipeline

github.com/taiyo130/docker-lesson-2569

ตะวัน เสริมสุข · วิศวกรรมคอมพิวเตอร์ · [10/10/2569]

ตารางวันนี้

วันที่ 1 — Docker Fundamentals

เวลา	หัวข้อ	Lab
09:00–09:45	Intro: VM vs Container, Architecture	—
09:45–10:30	Docker CLI & Container Lifecycle	Lab 1
10:30–10:45	พักเบรก	
10:45–12:00	Dockerfile แรก	Lab 2
12:00–13:00	พักกลางวัน	
13:00–14:30	Build อย่างมืออาชีพ: cache, multi-stage, push	Lab 3
14:30–14:45	พักเบรก	
14:45–16:00	Data: Volume, Bind mount, ENV	Lab 4
16:00–17:00	Mini-project: containerize React	Lab 5

ปลายทางของ 2 วัน

git push ครั้งเดียว → เว็บขึ้นเอง

1 **git push** ▷ 2 **test** ▷ 3 · วันนี้ **build image** ▷ 4 · วันนี้ **push registry** ▷ 5 **deploy**

วันนี้เราสร้าง “ชิ้นส่วน” ที่ pipeline ต้องใช้: **image ที่เล็ก, build เร็ว, ปลอดภัย** และขึ้น registry ได้ พรุ่งนี้ให้ GitHub Actions ทำแทนเรา แล้ว deploy ขึ้น stdXX.cpedev.cloud

INTRO

“เครื่องผมรันได้นะ”

ประโยคที่ทุกทีมเคยได้ยิน — และเหตุผลที่ Docker เกิดขึ้น

ปัญหาแบบดั้งเดิม

โค้ดเหมือนกัน แต่ “สภาพแวดล้อม” ไม่เหมือนกัน

Runtime คนละ version

เครื่องเรา Node 24 · server Node 18 · เพื่อน Node 20

OS และ library ต่างกัน

Windows vs Linux, path, line ending, native module

Config กระจัดกระจาย

port, รหัส DB, env ที่ “จำได้ว่าเคยตั้ง”

คู่มือติดตั้ง 20 หน้า

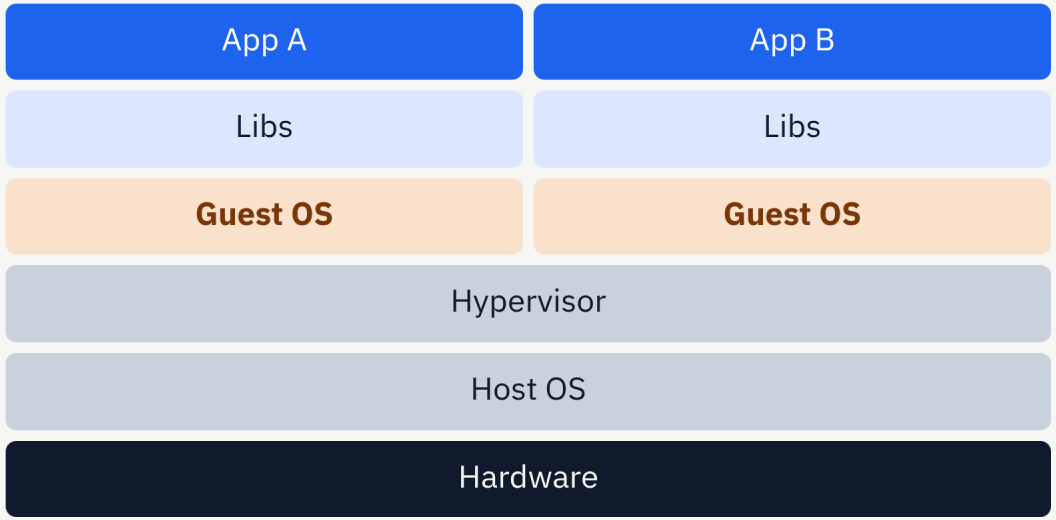
คนใหม่เข้าทีม เสียเวลา setup ทั้งวัน

แนวคิดของ Docker: แพ็กแอป + ทุกอย่างที่แอปต้องใช้ ไว้ในกล่องเดียว แล้วรันได้เหมือนกันทุกที่

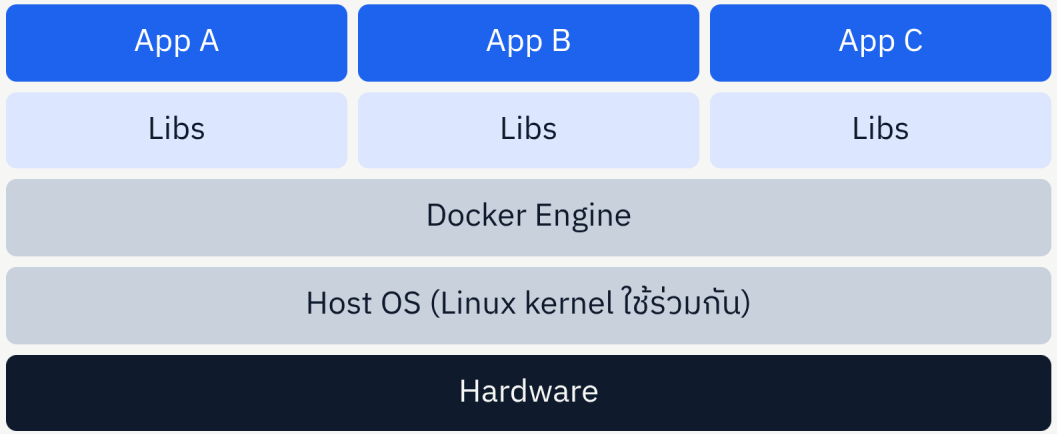
VM VS CONTAINER

Container ไม่มี Guest OS ของตัวเอง

Virtual Machine



Container



VM VS CONTAINER

เทียบกันตรง ๆ

	Virtual Machine	Container
Virtualize ระดับ	Hardware (Hypervisor)	OS (kernel ใช้ร่วมกัน)
เวลา start	เป็นนาที (boot OS)	เป็นวินาทีหรือเร็วกว่า
ขนาด	หลาย GB	หลัก MB ถึงหลักร้อย MB
ความหนาแน่น	หลักสิบ VM ต่อเครื่อง	หลักร้อย container ต่อเครื่อง
Isolation	แข็งแรงกว่า (kernel แยก)	เบากว่า (kernel เดียวกัน)
เหมาะกับ	คนละ OS, งานที่ต้องแยกเด็ดขาด	แพ็กแอป, microservices, CI/CD

ใช้ร่วมกันได้: server ส่วนใหญ่ในโลกจริงคือ **container ที่รันอยู่บน VM** (เช่น VPS ที่เราจะ deploy พรุ่งนี้)

ใต้ฝากระโปรง

Container = process ที่ถูก kernel “แยกมุมมอง”

Namespaces

เห็นอะไรได้บ้าง: pid, net, mnt, uts, user — process คิดว่าตัวเองอยู่คนเดียวในเครื่อง

cgroups

ใช้ได้เท่าไร: จำกัด CPU, RAM, I/O ต่อ container

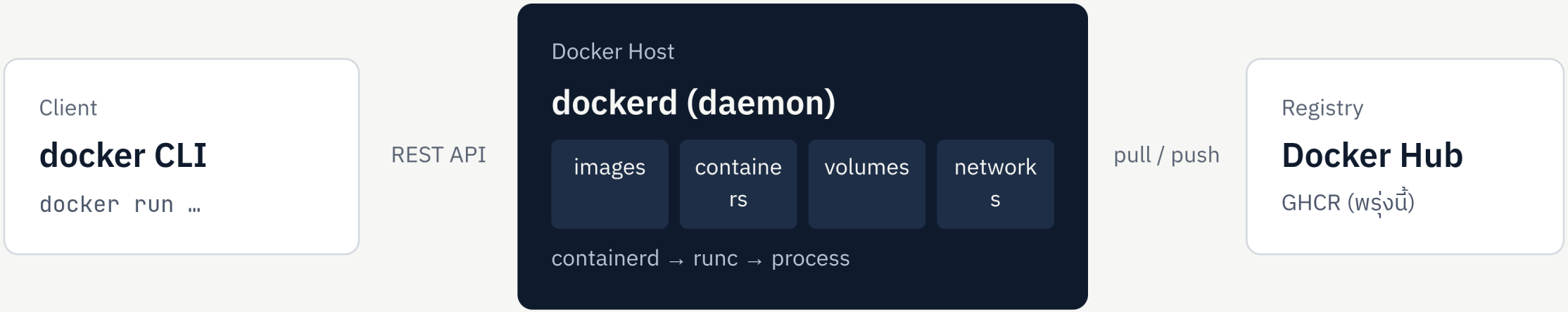
Union filesystem

ไฟล์มาจากไหน: image หลาย layer ซ้อนกัน (overlay2)

💡 Docker Desktop บน Windows/macOS แอบรัน **Linux VM เล็ก ๆ** (WSL2) ไว้ข้างหลัง เพราะ container ต้องใช้ Linux kernel

DOCKER ARCHITECTURE

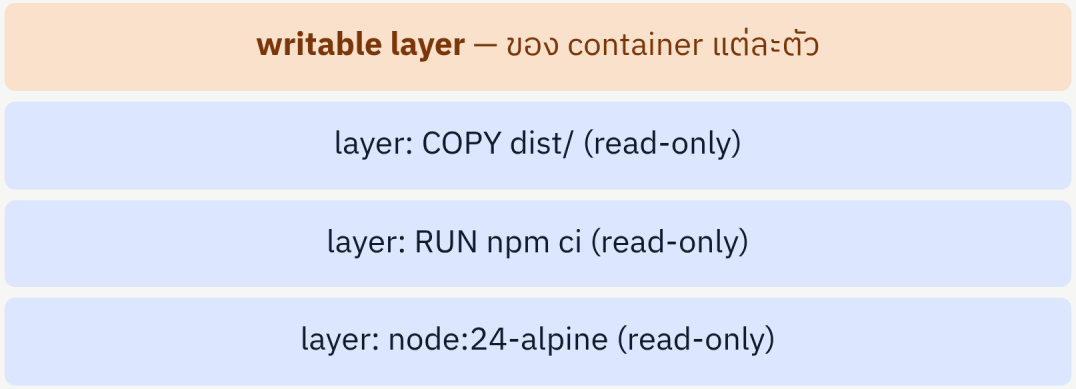
พิมพ์คำสั่งที่หนึ่ง ทำงานอีกที่หนึ่ง



คำสั่ง **docker version** แสดงทั้ง Client และ Server — ถ้าเห็นแค่ Client แปลว่า daemon ยังไม่ทำงาน

คำศัพท์ที่ต้องแยกให้ออก

Image คือ class · Container คือ object



Image

แม่แบบแบบอ่านอย่างเดียว ประกอบจากหลาย layer — แชร์กันได้

Container

image ที่กำลังรัน + writable layer บาง ๆ — สร้างกี่ตัวก็ได้จาก image เดียว

ลบ container = writable layer หายไปด้วย → เหตุผลที่ต้องมี **Volume** (Lab 4)

LAB 1 · 09:45–10:30

Docker CLI & Lifecycle

รัน · ดู · เข้าไปข้างใน · หยุด · ลบ

`labs/day1/01-cli/README.md`

LAB 1 · คำสั่งแรก

ผ่าคำสั่ง docker run

```
docker run -d --name web -p 8080:80 nginx:1.29-alpine
```

ส่วน	ความหมาย
-d	detached — รันเบื้องหลัง คืน terminal ให้เรา
--name web	ตั้งชื่อ (ไม่ตั้ง Docker จะสุ่มชื่อให้)
-p 8080:80	port เครื่องเรา : port ใน container
nginx:1.29-alpine	image ชื่อ:tag — ถ้ายังไม่มีในเครื่อง จะ pull ให้เอง

LAB 1 · ดูข้างใน

4 คำสั่งที่ใช้ debug ทุกวัน

`docker logs -f web`

ดู stdout/stderr ของแอป (แอปควร log ออก stdout ไม่ใช่ไฟล์)

`docker exec -it web sh`

เปิด shell ใน container ที่รันอยู่ — ลอง ps, ls, cat

`docker inspect web`

ทุกอย่างเป็น JSON: IP, mounts, env, restart count

`docker stats`

CPU / RAM / network ของแต่ละ container แบบ real-time

LAB 1 · LIFECYCLE

วงจรชีวิตของ container



docker run = create + start ในคำสั่งเดียว

docker stop ส่ง SIGTERM → รอ 10 วิ → SIGKILL

docker ps -a เห็น container ที่ Exited ด้วย

docker rm -f = stop + rm ทันที

LAB 1 · คำถามชวนคิด

curl ที่ติดตั้งไว้หายไปไหน?

```
$ docker run -it --rm alpine:3 sh
/ # apk add curl
/ # curl -I https://www.google.com ✓ ใช้ได้
/ # exit
$ docker run -it --rm alpine:3 sh -c "which curl || echo หายแล้ว"
หายแล้ว
```

ทุก container เริ่มจาก image เดิมเสมอ — **สิ่งที่ต้องติดตั้ง ให้เขียนลงใน Dockerfile** ไม่ใช่ติดตั้งมือใน container

LAB 2 · 10:45–12:00

Dockerfile แรก

สูตรอาหารสำหรับสร้าง image ที่ทำซ้ำได้ทุกครั้ง

`labs/day1/02-first-dockerfile/`

LAB 2 · DOCKERFILE

6 บรรทัด ได้ image ของเราเอง

```
FROM node:24-alpine
WORKDIR /app
COPY server.js .
ENV PORT=3000
EXPOSE 3000
CMD ["node", "server.js"]
```

FROM เริ่มจาก image ที่มี Node แล้ว

WORKDIR cd เข้าโฟลเดอร์ (สร้างให้)

COPY จากเครื่องเรา → image

ENV ค่า default ของ env

EXPOSE เอกสาร: แอปใช้ port นี้

CMD คำสั่งตอน container เริ่ม

```
docker build -t hello-node:1.0 .
docker run -d -p 3000:3000 -e STUDENT_NAME=สมชาย hello-node:1.0
```

LAB 2 · DOCKER BUILD

จุด “.” ท้ายคำสั่งคือ build context

```
docker build -t hello-node:1.0 .
```

-t ชื่อ:tag

ตั้งชื่อ image ไม่ใส่ tag จะได้ **latest**

. (context)

โฟลเดอร์ที่ถูกส่งให้ daemon — **COPY อ่านได้แค่ไฟล์ในนี้**

docker history

ดูว่าแต่ละคำสั่งสร้าง layer ขนาดเท่าไร

LAB 2 · คำสั่งหลัก

ทำงานตอน build หรือตอน run?

คำสั่ง	ทำงานตอน	ใช้ทำอะไร
FROM	build	base image (และตั้งชื่อ stage ด้วย AS)
RUN	build	รันคำสั่ง → ผลลัพธ์กลายเป็น layer ใหม่
COPY	build	ไฟล์จาก context หรือจาก stage อื่น
WORKDIR	build + run	โฟลเดอร์ทำงาน
ENV	build + run	environment variable (override ได้ด้วย -e)
USER	build + run	รันด้วย user ไหน (อย่าใช้ root)
EXPOSE	— (เอกสาร)	บอกว่าแอปใช้ port อะไร
CMD / ENTRYPOINT	run	คำสั่งตอน container เริ่มทำงาน

LAB 2 · CMD VS ENTRYPOINT

ใครถูกแทนที่ได้ ใครไม่ได้

CMD — ค่า default

```
CMD ["node", "server.js"]

# argument ท้ายคำสั่ง แทน CMD ทั้งหมด
docker run hello-node:1.0 node --version
```

ENTRYPOINT — ตัวคำสั่งหลัก

```
ENTRYPOINT ["curl", "-s"]

# argument ต่อท้าย ENTRYPOINT
docker run mycurl https://example.com
```

⚠ ใช้ **exec form** ["node", "server.js"] เสมอ — shell form **CMD node server.js** รันผ่าน /bin/sh แอปจะไม่ได้รับ SIGTERM ตอน docker stop

LAB 3 · 13:00–14:30

Build อย่างมืออาชีพ

แอปจริง: Express + TypeScript (app/api)
image เล็ก · build เร็ว · ปลอดภัย · พร้อมขึ้น registry

`labs/day1/03-build-like-a-pro/`

LAB 3 · จับผิด

Dockerfile ที่ “ใช้ได้” แต่ไม่ดี

```
FROM node:24
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build
CMD ["npm", "start"]
```

1. **node:24** (Debian เต็ม) → image 1.31 GB
2. **COPY . .** ก่อน install → แก้โค้ดบรรทัดเดียว ติดตั้ง package ใหม่หมด
3. ไม่มี **.dockerignore** → node_modules ของเครื่องเราติดเข้าไป
4. devDependencies และ source ติดไปใน image ที่ใช้รันจริง
5. รันด้วย **root**

ลองก่อนเฉลย: copy เป็น app/api/Dockerfile แล้ว **docker build -t todo-api:naive .** และดู docker images

LAB 3 · LAYER CACHE

layer หนึ่งเปลี่ยน ทุก layer ข้างล่าง build ใหม่

```
FROM node:24-alpine · CACHED
WORKDIR /app · CACHED
COPY . . · เปลี่ยน! (แก้ server.ts)
RUN npm install · รันใหม่ทั้งหมด
RUN npm run build · รันใหม่
```

กฎข้อเดียวที่ต้องจำ

สิ่งที่**เปลี่ยนน้อย** (dependencies) ไว้บน
สิ่งที่**เปลี่ยนบ่อย** (source code) ไว้ล่าง

ใน CI ทุก commit = build ใหม่ 1 ครั้ง — cache ดีคือ pipeline เร็ว

LAB 3 · LAYER CACHE

สลับลำดับนิดเดียว cache ใช้ได้

ก่อน — cache พังทุกครั้ง

```
COPY . .
RUN npm install
RUN npm run build
```

หลัง — install ถูก cache

```
COPY package.json package-lock.json ./
RUN npm ci
COPY tsconfig.json ./
COPY src ./src
RUN npm run build
```

npm ci แทน npm install: ติดตั้งตาม package-lock.json เป๊ะ ๆ ทุกครั้ง — ผลลัพธ์เหมือนกันทุกเครื่อง ทุก pipeline

LAB 3 · .DOCKERIGNORE

อย่าส่งของไม่จำเป็นเข้า build

```
# app/api/.dockerignore
node_modules
dist
npm-debug.log
.env
.git
Dockerfile*
.dockerignore
```

node_modules

ของ Windows อาจมี native binary ที่รันบน Linux ไม่ได้ — ให้ npm ci ใน container ติดตั้งเอง

.env

รหัสผ่านไม่ควรติดไปใน image ที่ push ขึ้น registry

.git

ใหญ่ เปลี่ยนทุก commit และทำให้ cache ของ COPY . . พัง

LAB 3 · MULTI-STAGE BUILD

build ในห้องครัว เสิร์ฟแค่จานอาหาร

```
FROM node:24-alpine AS deps
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
FROM deps AS build
COPY tsconfig.json ./
COPY src ./src
RUN npm run build
FROM node:24-alpine AS runtime
ENV NODE_ENV=production
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY --from=build /app/dist ./dist
USER node
CMD ["node", "dist/server.js"]
```

deps

ติดตั้งทุก package รวม typescript — ทิ้ง

build

tsc → dist/ — เอาไปแค่ dist/

runtime ✓

prod deps + dist/ + USER node — นี่คือ image ที่ได้

ผลลัพธ์ · วัดจริง

naive

1.31 GB



multi-stage

175 MB

เล็กลงประมาณ 87% — pull เร็วขึ้น deploy เร็วขึ้น ช่องโหว่น้อยลง

todo-api บน Docker 29 · node:24 เทียบ node:24-alpine

LAB 3 · เลือก BASE IMAGE

เล็กที่สุด ไม่ได้แปลว่าดีที่สุดเสมอ

Tag	ขนาด	เหมาะกับ
node:24	1.14 GB	stage ที่ต้อง compile native module (มี gcc, python)
node:24-slim	230 MB	Debian แบบตัดของ — ใช้ glibc เข้ากันได้กว้างที่สุด
node:24-alpine	171 MB	เล็กสุด แต่ใช้ musl — library บางตัวอาจต้อง build เอง

ระบุ version เสมอ

node:24-alpine ไม่ใช่ node:latest — build วันนี้กับเดือนหน้าต้องได้ผลเหมือนกัน

ใช้ official image

มีป้าย Docker Official Image บน Hub ได้รับ security update สม่ำเสมอ

LAB 3 · SECURITY & PRODUCTION

4 บรรทัดที่ image production ควรมี

`USER node`

ถ้าแอปโดนเจาะ ผู้บุกรุกไม่ได้สิทธิ์ root ใน container

`ENV NODE_ENV=production`

Express และ library ต่าง ๆ ปิดโหมด debug

`HEALTHCHECK CMD wget …`

docker ps แสดง (healthy) — Compose ใช้รอ service พร้อม (วันที่ 2)

`process.on("SIGTERM")`

ปิดแอปอย่างนุ่มนวลตอน deploy ตัวใหม่ (อยู่ใน src/server.ts)

ห้าม ใส่รหัสผ่านใน Dockerfile (ENV DB_PASSWORD=...) — ใครได้ image ไป docker history ก็อ่านได้

LAB 3 · TAG & PUSH

ชื่อ image เต็ม ๆ บอกว่าอยู่ที่ไหน

```
docker.io/somchai/todo-api:1.0
```

registry · user · ชื่อ · tag — พรุ่งนี้: ghcr.io/<user>/<repo>-api:<commit-sha>

```
docker login
docker tag todo-api:1.0 somchai/todo-api:1.0
docker push somchai/todo-api:1.0
```

latest ไม่ได้แปลว่าล่าสุด — เป็นแค่ชื่อ tag default ที่ถูกเขียนทับได้ตลอด

production ใช้ tag ที่ไม่เปลี่ยน: **1.0.3** (semver) หรือ **a1b2c3d** (git SHA)

LAB 4 · 14:45–16:00

Data ที่รอดชีวิต

Volume · Bind mount · Environment variables

`labs/day1/04-data/`

LAB 4 · ปัญหา

ลบ container = ข้อมูลหายตาม

```
docker run -d --name api -p 3000:3000 todo-api:1.0
irm localhost:3000/api/todos -Method Post -ContentType "application/json" `
  -Body '{"title":"will it survive?"}'
docker rm -f api
docker run -d --name api -p 3000:3000 todo-api:1.0
irm localhost:3000/api/todos  # ว่างเปล่า — หายหมด
```

ทำไม?

ข้อมูลอยู่ใน RAM / writable layer ของ container ตัวนั้น

แล้ว deploy ล่ะ?

ทุก deploy = สร้าง container ใหม่ — database ต้องอยู่นอก container

LAB 4 · ทางออก

Named Volume vs Bind Mount

Named Volume

`-v pgdata:/var/lib/postgresql`

Docker จัดการที่เก็บให้ · ย้ายเครื่องง่าย · ไม่ขึ้นกับ path ของ host

ใช้กับ: database, production

Bind Mount

`-v "${PWD}/site:/usr/share/nginx/html"`

โฟลเดอร์จริงในเครื่องเรา · แก้ไฟล์แล้วเห็นใน container ทันที

ใช้กับ: development, hot reload

LAB 4 · ทดลอง

ลบ container แล้วข้อมูลยังอยู่

```
docker volume create pgdata
docker run -d --name db -e POSTGRES_PASSWORD=secret `
  -v pgdata:/var/lib/postgresql postgres:18-alpine
docker exec -it db psql -U postgres  # CREATE TABLE notes … ; INSERT …
docker rm -f db  # ลบ container ทิ้ง
docker run …  # คำสั่งเดิม ใช้ volume pgdata เดิม
docker exec db psql -U postgres -c "SELECT * FROM notes"  # ยังอยู่ ✓
```

⚠ **Postgres 18 เปลี่ยน path** เป็น /var/lib/postgresql — บทความเก่าที่ใช้ .../postgresql/data จะ start ไม่ขึ้น

LAB 4 · BIND MOUNT

แก้ไฟล์บนเครื่อง เห็นผลใน container ทันที

```
cd labs/day1/04-data
docker run -d --name site -p 8080:80 `
  -v "${PWD}/site:/usr/share/nginx/html:ro" nginx:1.29-alpine
```

${PWD}

ใช้ได้ทั้ง bash และ PowerShell — ใส่ " " ครอบเสมอ (path มีช่องว่าง)

:ro

read-only — container อ่านได้อย่างเดียว แก้ไฟล์ของเราไม่ได้

` ท้ายบรรทัด

PowerShell ขึ้นบรรทัดใหม่ด้วย backtick (macOS/Linux ใช้ \ แทน)

LAB 4 · ENVIRONMENT

Config อยู่นอก image · Secret อยู่นอก git

```
# db.env
POSTGRES_PASSWORD=secret
POSTGRES_DB=todo

docker run -d --env-file db.env `
  postgres:18-alpine
```

image เดียว ใช้ได้ทุกที่ — dev, test, production ต่างกันแค่ env

.env ต้องอยู่ใน **.gitignore** เสมอ (repo นี้ตั้งไว้แล้ว)

พรุ่งนี้: secret ใน CI เก็บที่ **GitHub Secrets** ไม่ใช่ในโค้ด

LAB 5 · 16:00–17:00 · MINI-PROJECT

Containerize React

ทำเองทั้งหมด: production image · คุยกับ api · dev container

`labs/day1/05-mini-project/` · เฉลย `solutions/day1/web/`

LAB 5 · ส่วนที่ 1–2

React ไม่ต้องการ Node ตอนรันจริง



Build-time config

ตัวแปร VITE_* ถูกฝังลงไฟล์ JS ตอน vite build — เปลี่ยนตอนรันไม่ได้

Runtime config

API_UPSTREAM ใน nginx template — เปลี่ยนด้วย -e ได้ image เดียว ใช้ได้ทุกที่

LAB 5 · ส่วนที่ 3

Dev container + hot reload

```
docker run --rm -p 5173:5173 `
  -v "${PWD}:/app" -v /app/node_modules `
  -e VITE_USE_POLLING=true `
  -e API_URL=http://host.docker.internal:3000 `
  --add-host=host.docker.internal:host-gateway `
  todo-web:dev
```

flag	ทำไมต้องมี
-v "${PWD}:/app"	bind mount โค้ดเข้าไป
-v /app/node_modules	กัน node_modules ของเครื่องเราทับของใน image
VITE_USE_POLLING=true	Windows ไม่ส่ง file event ผ่าน bind mount — ให้ Vite ตรวจเอง
host.docker.internal	ชื่อที่ชี้กลับมาที่เครื่องเรา (api ที่ port 3000)

LAB 5 · เกณฑ์ผ่าน

เช็กให้ครบ 4 ข้อก่อนกลับบ้าน

- **todo-web:1.0** เล็กกว่า 100 MB และเปิดหน้าเว็บได้
- เพิ่ม / ลบ todo ผ่านหน้าเว็บได้ (web ↔ api ผ่าน network todo-net)
- แก้ src/App.tsx แล้วเห็นผลทันทีใน dev container
- push **todo-api** และ **todo-web** ขึ้น Docker Hub ของตัวเอง

TROUBLESHOOTING

Error ที่จะเจอวันนี้ และวิธีแก้

Error	สาเหตุ / วิธีแก้
port is already allocated	มี container อื่นใช้ port นั้น — docker ps แล้วเปลี่ยนเลขฝั่งซ้ายของ -p
Cannot connect to the Docker daemon	Docker Desktop ยังไม่ start — เปิดแล้วรอไอคอนเป็นสีเขียว
Conflict. The container name is already in use	ชื่อซ้ำกับตัวที่ Exited — docker rm -f ชื่อนั้น
COPY ... "/xxx": not found	ไฟล์อยู่นอก build context หรือโดน .dockerignore
toomanyrequests (pull)	ติด rate limit ของ Docker Hub — docker login
no space left on device	docker system df แล้ว docker system prune

สรุปวันที่ 1

วันนี้ทำอะไรไปแล้ว · พรุ่งนี้ไปต่อที่ไหน

ทำได้แล้ว

- run / logs / exec / inspect / rm
- Dockerfile, layer cache, .dockerignore
- multi-stage: 1.31 GB → 175 MB
- non-root, healthcheck, SIGTERM
- volume, bind mount, env-file
- tag & push ขึ้น Docker Hub

พรุ่งนี้

- ทำไม nginx รู้จักชื่อ “api”? → Networking
- คำสั่งยาว ๆ วันนี้ → compose.yaml ไฟล์เดียว
- web + api + PostgreSQL
- GitHub Actions: test → build → push GHCR
- deploy ขึ้น stdXX.cpedev.cloud

จบวันที่ 1

Q & A

ก่อนกลับ: **docker system prune** · fork repo · commit Dockerfile ของตัวเอง

github.com/taiyo130/docker-lesson-2569 · cheatsheet.md